Plano de Testes – Bibliotech

Projeto Integrador – Etapa 7: testes unitários com JUnit e plano de testes manuais

Gabriel Marostica

Outubro de 2026

Curso / turma: [preencher] **Unidade curricular:** [preencher] **Professor(a):** [preencher]

Repositório: [colar aqui o link do repositório no GitHub]

1. Objetivo

Este plano define como o Bibliotech (sistema de gerenciamento de biblioteca escolar) será testado, para verificar se os requisitos já implementados no núcleo do sistema, e os que serão exibidos na versão web, funcionam como esperado e continuam funcionando depois de cada alteração.

O plano cobre dois tipos de teste:

- **Testes automatizados (JUnit 4):** já implementados e executados no projeto `BibliotechTestes`.
- **Testes manuais:** casos para executar na interface do sistema web (planejada para a próxima fase) e contra o banco MySQL real.

2. Escopo

Dentro do escopo	Fora do escopo desta etapa
Regras de negócio: empréstimo, estoque, devolução, atraso, multa, login	Aparência e usabilidade visual das telas (o sistema web ainda não existe)
Validação de dados das entidades (aluno, livro, funcionário, usuário)	Desempenho e testes de carga
Segurança básica de senha	Acesso ao banco nos testes unitários (dispensado pela atividade; verificado de outra forma, veja a seção 4)
Integridade do estoque em operações combinadas	Testes de compatibilidade entre navegadores

3. Requisitos a serem testados

Os requisitos vêm das funcionalidades descritas no `README` do sistema desktop e das regras de negócio que surgiram na refatoração.

ID	Requisito	Prioridade
RF01	Login de usuário com senha	Alta
RF02	Cadastrar, consultar, atualizar, excluir e buscar alunos	Alta
RF03	Cadastrar e listar funcionários	Média
RF04	Cadastrar, consultar, excluir e buscar livros (com controle de quantidade)	Alta
RF05	Registrar empréstimo de um ou mais	Alta

ID	Requisito	Prioridade
	livros a um aluno	
RF06	Controlar o estoque: baixa no empréstimo e retorno na devolução	Alta
RF07	Registrar devolução de um empréstimo	Alta
RF08	Calcular multa por dia de atraso	Alta
RF09	Listar empréstimos pendentes de devolução	Média
RNF01	Senhas guardadas com hash (nunca em texto puro)	Alta
RNF02	Credenciais do banco fora do código-fonte	Alta
RNF03	Integridade: um empréstimo ou devolução é gravado por inteiro ou não é gravado	Alta
RNF04	Mensagens de erro claras para o usuário	Média

4. Estratégia de teste

Nível	O que testa	Como	Estado
Unitário	Regras de uma classe isolada (cálculo de multa, dias de atraso, estoque do livro, validações)	JUnit 4, sem banco	**Implementado**
Integração em memória	Casos de uso completos (serviço + repositório), com um relógio controlável	JUnit 4 com repositórios em memória	**Implementado**
Integração com banco	Repositórios JDBC e transações no MySQL	`Main` do projeto principal, com `BIBLIOTECH_DB_*` configurado	Implementado; executado em H2 (modo MySQL) e a executar no MySQL
Manual / sistema	Fluxos do usuário na interface web	Casos da seção 7	A executar quando a interface existir

Por que os testes unitários não acessam o banco: os serviços recebem os repositórios como interfaces (injeção de dependência). Nos testes usam-se implementações em memória, o que os torna rápidos e repetíveis. O relógio também é injetado (`java.time.Clock`), o que permite testar "devolução com 3 dias de atraso" sem esperar três dias.

Separação de responsabilidade para testar: o cálculo de atraso e o cálculo da multa já estavam isolados em métodos próprios (`DevolucaoServico.calcularDiasAtraso` e `PoliticaMulta.calcular`), exatamente para poderem ser testados sem tela e sem banco.

5. Ambiente e ferramentas

Item	Versão / detalhe
Linguagem	Java 21
IDE / build	NetBeans (projeto Ant)
Framework de testes	JUnit 4.13.2 + Hamcrest 2.2 (incluídos em `BibliotechTestes/lib`)
Banco de dados	MySQL 8 (`banco/schema.sql`)
Controle de versão	Git / GitHub

Como executar os testes automatizados: no NetBeans, abra o projeto `BibliotechTestes`, clique com o botão direito e escolha **Test** (Alt+F6). Pelo terminal, dentro da pasta do projeto: `ant test`.

6. Testes automatizados (JUnit)

6.1 Resultado da execução

Execução de 21/10/2026, com `ant test`: **85 testes, 0 falhas, 0 erros.**

Classe de teste	Testes	Falhas	Foco
`MultaPorDiaFixoTest`	9	0	multa com e sem atraso, valor configurável, arredondamento, troca de estratégia
`CalculoDiasAtrasoTest`	7	0	dias de atraso no prazo, atrasado, antecipado, mudança de mês, de ano e ano bissexto
`LivroTest`	9	0	baixa e retorno de estoque, estoque insuficiente, validação de dados
`EmprestimoTest`	9	0	datas, livro duplicado, lista de itens protegida, resumo
`EntidadesValidacaoTest`	9	0	dados obrigatórios de aluno, funcionário, usuário e devolução; classe `Validacao`
`CodificadorSenhaPbkdf2Test`	6	0	hash com sal, senha certa e errada, hash inválido
`EmprestimoServicoTest`	12	0	empréstimo, baixa de estoque, atomicidade, erros, pendentes
`DevolucaoServicoTest`	7	0	devolução no prazo e atrasada, multa, retorno ao estoque, devolução duplicada
`AutenticacaoServicoTest`	7	0	login, senha errada, usuário inexistente, cadastro
`AlunoServicoTest`	10	0	cadastro, matrícula duplicada, atualização, exclusão, busca
Total	**85**	**0**	

6.2 Rastreabilidade: requisito × testes automatizados

Requisito	Testes que o cobrem	Cobertura
RF01 Login	`AutenticacaoServicoTest`, `CodificadorSenhaPbkdf2Test`	Total
RF02 Alunos	`AlunoServicoTest`, `EntidadesValidacaoTest`	Total
RF03 Funcionários	`EntidadesValidacaoTest` (validação); cadastro e listagem pelo `Main`	Parcial
RF04 Livros	`LivroTest`; cadastro e busca pelo `Main`	Parcial
RF05 Empréstimo	`EmprestimoTest`,	Total

Requisito	Testes que o cobrem	Cobertura
	`EmprestimoServicoTest`	
RF06 Estoque	`LivroTest`, `EmprestimoServicoTest`, `DevolucaoServicoTest`	Total
RF07 Devolução	`DevolucaoServicoTest`, `CalculoDiasAtrasoTest`	Total
RF08 Multa	`MultaPorDiaFixoTest`, `CalculoDiasAtrasoTest`, `DevolucaoServicoTest`	Total
RF09 Pendentes	`EmprestimoServicoTest`	Total
RNF01 Hash de senha	`CodificadorSenhaPbkdf2Test`, `AutenticacaoServicoTest`	Total
RNF02 Credenciais fora do código	Somente manual (CT-M16)	Manual
RNF03 Integridade	`EmprestimoServicoTest` (pedido misto recusado por inteiro); `Main` com JDBC	Parcial
RNF04 Mensagens claras	Verificadas manualmente na interface (CT-M02, M04, M05, M11)	Manual

6.3 Os testes encontram erros de verdade?

Para conferir a sensibilidade dos testes, foram introduzidos dois erros propositais no código, em uma cópia: o cálculo de atraso deixou de limitar o resultado a zero, e a checagem de estoque passou a aceitar um exemplar a mais do que existe. Resultado: **4 classes de teste falharam** (`CalculoDiasAtrasoTest`, `LivroTest`, `EmprestimoServicoTest`, `DevolucaoServicoTest`). Depois o código foi restaurado, e os 85 testes voltaram a passar.

6.4 Bugs do sistema desktop que os testes agora protegem

Bug da versão desktop	Teste que impede a volta
Atraso calculado em dias desde 1970 (multa de dezenas de milhares de reais)	`CalculoDiasAtrasoTest.atrasoNuncaEAbsurdoComoNaVersaoDesktop` e `DevolucaoServicoTest.tresDiasDeAtrasoGeramMultaDeSeisReais`
Estoque baixado só em memória e fora do controle	`EmprestimoServicoTest.registrarEmprestimoGravaEBaixaOEstoque`
`adicionarItem` baixava 1 exemplar, ignorando a quantidade	`EmprestimoTest.adicionarItemNaoMexeNoEstoque`
Falha de estoque era só um `println`, e a operação continuava	`LivroTest.emprestarMaisDoQueOEstoqueLancaExcecaoENaoAlteraNada`
Dados inválidos aceitos (nome vazio, quantidade negativa)	`EntidadesValidacaoTest`, `LivroTest`

7. Casos de teste manuais

Os casos abaixo valem para a interface web planejada e para o banco MySQL. A coluna *Resultado obtido* deve ser preenchida na execução (**OK** ou **FALHA** com a descrição).

ID	Req.	Passos	Resultado esperado	Resultado obtido
CT-M01	RF01	Abrir a página de login. Informar um usuário e senha válidos e entrar.	Abre a página principal do sistema.	
CT-M02	RF01, RNF04	Informar login válido com senha errada;	Nos dois casos aparece “Login ou	

ID	Req.	Passos	Resultado esperado	Resultado obtido
		repetir com login inexistente.	senha inválidos.”, sem indicar qual dado está errado.	
CT-M03	RF01, RNF01	Observar o campo de senha. Tentar abrir uma página interna sem ter feito login.	A senha aparece oculta (pontos). O acesso sem login volta para a tela de login.	
CT-M04	RF02, RNF04	Cadastrar aluno deixando o nome em branco.	Mensagem de campo obrigatório. Nada é gravado.	
CT-M05	RF02, RNF04	Cadastrar dois alunos com a mesma matrícula.	O segundo é recusado com mensagem de matrícula duplicada.	
CT-M06	RF02	Buscar um aluno digitando parte do nome em letras maiúsculas.	A lista mostra os alunos que contêm o trecho, sem diferenciar maiúsculas e minúsculas.	
CT-M07	RF02	Excluir um aluno sem empréstimos. Depois tentar excluir um aluno que já tem empréstimo.	O primeiro some da lista. O segundo mostra uma mensagem de erro e o sistema continua funcionando.	
CT-M08	RF03	Cadastrar um funcionário e abrir a listagem.	O funcionário aparece na lista (na versão desktop ele não era gravado).	
CT-M09	RF04	Cadastrar livro com quantidade negativa; repetir com ano igual a 0.	Os dois cadastros são recusados com mensagem clara.	
CT-M10	RF05, RF06	Com um livro de 3 exemplares, registrar empréstimo de 2. Conferir a listagem de livros.	Empréstimo registrado. O livro passa a mostrar 1 exemplar.	
CT-M11	RF06, RNF04	Tentar emprestar mais exemplares do que o estoque.	Mensagem de estoque insuficiente. Nenhum estoque é alterado e nenhum empréstimo é criado.	
CT-M12	RF05	Registrar empréstimo com data prevista de devolução anterior a hoje.	O empréstimo é recusado.	
CT-M13	RF07, RF08	Registrar um empréstimo e, com a data do sistema 3 dias depois do prazo, devolvê-lo.	A tela mostra 3 dias de atraso e multa de R$ 6,00. O estoque volta ao valor anterior.	
CT-M14	RF07	Tentar devolver o mesmo empréstimo duas vezes.	A segunda tentativa é recusada (“já foi devolvido”). O estoque não aumenta de novo.	
CT-M15	RF09	Abrir a lista de pendentes antes e	O empréstimo aparece como	

ID	Req.	Passos	Resultado esperado	Resultado obtido
		depois de devolver um empréstimo.	pendente e some depois da devolução.	
CT-M16	RNF02	Executar o sistema sem definir `BIBLIOTECH_DB_PASSWORD`.	O sistema avisa que a senha do banco não está configurada. Não existe senha escrita no código.	
CT-M17	RNF03	Executar o `Main` com `BIBLIOTECH_DB_URL`, `_USER` e `_PASSWORD` apontando para o MySQL com o `schema.sql` aplicado.	Todas as verificações passam (79 de 79, incluindo as do banco).	
CT-M18	RNF03	Com o sistema aberto, parar o MySQL e tentar registrar um empréstimo. Subir o MySQL de novo e consultar.	O sistema mostra erro de banco, sem derrubar a aplicação. Nenhum empréstimo parcial foi gravado.	

8. Critérios de aceitação

- **Entrada:** código compilando; banco criado com `banco/schema.sql` (para os testes com banco).
- **Saída:** 100% dos testes automatizados passando; todos os casos manuais de prioridade Alta com resultado **OK**; nenhum defeito de prioridade Alta aberto.
- **Registro de defeitos:** cada falha encontrada é anotada com ID do caso, passos para reproduzir, resultado obtido e resultado esperado, e vira uma *issue* no GitHub.
- **Regressão:** a suíte JUnit deve ser executada (`ant test`) antes de cada *commit* que altere regras de negócio.

9. Riscos e limitações

- As classes que acessam o banco (`*Jdbc`) não têm testes unitários, conforme a atividade permite. Elas foram verificadas pelo `Main` contra um banco H2 em modo de compatibilidade com MySQL (79 de 79 verificações). O caso CT-M17 confirma no MySQL real.
- A interface web ainda não existe, então os casos manuais ainda não foram executados.
- A cobertura de código não foi medida; a rastreabilidade da seção 6.2 mostra a cobertura por requisito.
- `FuncionarioServico` e `LivroServico` ainda não têm classe de teste própria; ficam como próximo passo.

10. Evidência de versionamento

O projeto de testes está versionado no repositório do sistema (`BibliotechTestes/`). Commits desta etapa:

Commit	Data	Mensagem
`310eace`	21/10/2026	chore: cria projeto NetBeans BibliotechTestes com JUnit 4
`e78ea1c`	21/10/2026	test: testes unitários do cálculo de atraso e da multa por dia

Commit	Data	Mensagem
`625b52a`	21/10/2026	test: testes das regras das entidades (Livro, Emprestimo) e do hash de senha
`4aa406a`	21/10/2026	test: testes dos casos de uso de empréstimo, devolução, login e alunos
`e44c867`	21/10/2026	docs: README com instruções do projeto de testes

Inclua abaixo as capturas de tela do repositório no GitHub:

1. **Figura 1:** pasta `BibliotechTestes` no GitHub, mostrando `test/`, `lib/` e `nbproject/`. [inserir captura]
2. **Figura 2:** aba *Commits*, mostrando os commits da etapa 7. [inserir captura]
3. **Figura 3:** execução dos testes no NetBeans (janela *Test Results* com os 85 testes aprovados). [inserir captura]